

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: INTERNET PROGRAMMING**

**COURSE CODE: CSA4305**

### **COURSE OUTCOME COVERED**

**CO4:** Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

### **QUESTIONS: Total Marks: 25**

#### **1: XML Parsing Algorithm**

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

ALGORITHM ParseXMLDocument

INPUT: xml\_file\_path (String)

OUTPUT: Formatted elements printed to console

TRY

Initialize xml\_parser = Create new XMLParser()

Initialize xml\_doc = xml\_parser.Load(xml\_file\_path)

CATCH Exception

PRINT "Error: Loading XML failed."

RETURN

END TRY

Initialize root\_node = xml\_doc.GetRootElement()

Call TraverseNodes(root\_node, 0)

END ALGORITHM

PROCEDURE TraverseNodes(current\_node, depth)

Initialize indent = Repeat(" ", depth)

IF current\_node.HasAttributes() THEN

FOR EACH attr IN current\_node.GetAttributes()

PRINT indent + "Attr: " + attr.GetName() + " = " + attr.GetValue()

END FOR

END IF

```

IF current_node.HasChildren() THEN
PRINT indent + "Node: " + current_node.GetName()
FOR EACH child IN current_node.GetChildren()
Call TraverseNodes(child, depth + 1)
END FOR
ELSE
PRINT indent + current_node.GetName() + ": " + current_node.GetTextValue() END IF
END PROCEDURE

```

## 2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

ALGORITHM MVC\_Workflow\_Handler

INPUT: user\_http\_request (HttpRequest)

OUTPUT: http\_response (HttpResponse)

// Step 1: Controller intercepting the request

Initialize controller = Router.RouteToController(user\_http\_request)

Initialize action\_parameters = controller.ExtractRequestParameters(user\_http\_request)

// Step 2: Controller interacts with Model

Initialize model\_object = controller.SelectModel(action\_parameters)

TRY

Initialize db\_connection = DatabaseManager.GetConnection()

Initialize query\_result =

db\_connection.ExecuteQuery(model\_object.GetSQLQuery(action\_parameters))

model\_object.PopulateState(query\_result)

CATCH DatabaseException

model\_object.SetErrorState("DB\_FAIL")

FINALLY

db\_connection.Close()

END TRY

// Step 3: Controller passes Model state to View and returns Response

Initialize view\_template = controller.SelectView(action\_parameters)

Initialize rendered\_html = view\_template.Render(model\_object.GetState()) Initialize

http\_response = Create new HttpResponse(rendered\_html)

RETURN http\_response

END ALGORITHM

## 3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server

processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

ALGORITHM JSP\_Request\_Response\_Flow

INPUT: client\_http\_request (HttpServletRequest)

OUTPUT: client\_http\_response (HttpServletResponse)

// Step 1: Client Request Initiation

Initialize jsp\_name = client\_http\_request.GetPath()

// Step 2: Translation and Compilation Check

IF NOT ServletContainer.IsCompiled(jsp\_name) OR ServletContainer.IsModified(jsp\_name)  
THEN

Initialize servlet\_java\_source = JSPCompiler.TranslateToServletSource(jsp\_name) Initialize  
servlet\_class = JavaCompiler.Compile(servlet\_java\_source)

ServletContainer.LoadServlet(servlet\_class)

END IF

// Step 3: Execution and Backend Interaction

Initialize servlet\_instance = ServletContainer.GetServletInstance(jsp\_name) Initialize  
jsp\_context = Create new JSPContext(client\_http\_request)

IF jsp\_context.RequiresDatabaseAccess() THEN

Initialize db\_conn = ConnectionPool.GetConnection()

jsp\_context.SetAttribute("pageData", db\_conn.FetchData(jsp\_context.GetQueryString()))  
db\_conn.Close()

END IF

// Step 4: Content Generation and Response Return

Initialize out\_stream = ServletContainer.GetOutputStream(client\_http\_response) Initialize  
rendered\_html = servlet\_instance.jspService(jsp\_context, out\_stream)

client\_http\_response.SetBody(rendered\_html)

client\_http\_response.Send()

END ALGORITHM

## 4: Database Retrieval

Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

ALGORITHM Database\_Retrieval\_Flow

INPUT: db\_credentials (DBConfig), query\_string (String)

OUTPUT: result\_set\_list (List of Objects)

Initialize db\_connection = NULL, prepared\_statement = NULL, result\_set = NULL Initialize  
result\_set\_list = Create empty List()

TRY

// Step 1: Establish Database Connection

db\_connection = DriverManager.getConnection(db\_credentials.GetURL(),

```

db_credentials.GetUsername(), db_credentials.GetPassword())
// Step 2: Prepare SQL Query
prepared_statement = db_connection.PrepareStatement(query_string) // Step 3: Execute SQL
Query
result_set = prepared_statement.ExecuteQuery()
// Step 4: Process the Result Set
WHILE result_set.Next() DO
Initialize record = Create new Map()
FOR EACH column IN result_set.GetMetaData().GetColumns() DO
record.Put(column.GetName(), result_set.GetObject(column.GetName())) END FOR
result_set_list.Add(record)
END WHILE
CATCH SQLException AS sql_error
PRINT "Database error: " + sql_error.GetMessage()
FINALLY
// Step 5: Close Resources
IF result_set != NULL THEN result_set.Close() END IF
IF prepared_statement != NULL THEN prepared_statement.Close() END IF IF db_connection !=
NULL THEN db_connection.Close() END IF
END TRY
RETURN result_set_list
END ALGORITHM

```

## 5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

ALGORITHM PHP\_User\_Input\_Validation

INPUT: post\_data (Array from \$\_POST)

OUTPUT: validation\_results (Map containing 'is\_valid' [Boolean] and 'errors' [List of Strings])

Initialize validation\_results = Create new Map()

Initialize errors = Create empty List()

validation\_results.Put("is\_valid", TRUE)

// Step 1: Extract and Sanitize Inputs

Initialize email\_input = TrimString(post\_data.Get("email"))

Initialize password\_input = post\_data.Get("password")

// Step 2: Check for Empty Fields

IF IsEmpty(email\_input) THEN

errors.Add("Email field cannot be empty.")

END IF

```

IF IsEmpty(password_input) THEN
errors.Add("Password field cannot be empty.")
END IF

// Step 3: Validate Email Format
IF NOT IsEmpty(email_input) THEN
IF NOT IsValidEmailPattern(email_input) THEN
errors.Add("Invalid email format.")
END IF
END IF

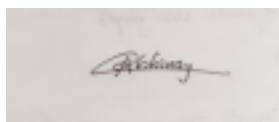
// Step 4: Validate Password Strength
IF NOT IsEmpty(password_input) THEN
IF Length(password_input) < 8 OR NOT HasUppercase(password_input) OR NOT
HasNumber(password_input) THEN
errors.Add("Password must be at least 8 characters, contain one uppercase letter, and one
number.")
END IF
END IF

// Step 5: Handle Validation Results
IF Length(errors) > 0 THEN
validation_results.Put("is_valid", FALSE)
validation_results.Put("errors", errors)
ELSE
validation_results.Put("is_valid", TRUE)
END IF
RETURN validation_results
END ALGORITHM

```

**Code of Conduct:**

*I \_\_\_\_G.Abhinay(192373062)\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



**Signature of the student:**

**RUBRICS FOR CALCULATION:**

| Criteria              | Weightage | Level 4 – Exemplary                                    | Level 3 – Proficient   | Level 2 – Developing  | Level 1 – Beginning      |
|-----------------------|-----------|--------------------------------------------------------|------------------------|-----------------------|--------------------------|
| Problem Understanding | 20%       | Identifies all inputs, outputs, constraints accurately | Minor missing elements | Partial understanding | Incorrect interpretation |

|                           |     |                                                                       |                                           |                              |                                 |
|---------------------------|-----|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Algorithm Design          | 30% | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20% | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10% | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20% | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |